

Workshop Guide: Build Your First AI Agent in 30 Minutes

Project: Universal Task Automator (Powered by Google Gemini)

Welcome to the **Nexaverse** AI Agents Workshop! In this session, you will transition from an AI user to an AI architect. We will use "**Vibe Coding**"—a rapid development approach using natural language and lightweight Python—to build a functional autonomous agent.

Phase 1: Setup Your Environment

We will use **Replit**, an online coding platform that requires zero installation.

1. Create a Repl:

- Go to [Replit.com](https://replit.com) and log in.
- Click + **Create Repl**, select the **Python** template, and name it `Gemini-Agent-Workshop`.

1. Get Your API Key:

- Go to [Google AI Studio](https://ai.google.dev/gemini-api/docs/api-key).
- Click **Get API Key** and copy your free-tier key.

1. Configure Secrets:

- In your Replit sidebar, find the **Tools** section and click **Secrets** (the padlock icon).
 - Add a new secret:
 - **Key:** `GEMINI_API_KEY`
 - **Value:** Paste your key here.

Phase 2: Define the "Anatomy" (The Files)

Create the following four files in your Replit project.

Step 1: The Dependencies (`requirements.txt`)

This file tells Replit which libraries to install.

Plain Text

```
google-generativeai
pydantic
streamlit
```

Step 2: The Schema (`schema.py`)

This defines the "shape" of the data our agent must return.

Python

```
from pydantic import BaseModel, Field
from typing import List

class AgentOutput(BaseModel):
    research_summary: List[str] = Field(description="Key points from the research")
    outreach_draft: str = Field(description="A personalized outreach message.")
```

Step 3: The Brain & Logic (`agent.py`)

This is where we define the agent's role and its ability to self-correct.

Python

```
import os
import json
import google.generativeai as genai
from pydantic import ValidationError
from schema import AgentOutput

class UniversalTaskAutomator:
    def __init__(self):
        genai.configure(api_key=os.environ.get("GEMINI_API_KEY"))
        self.model = genai.GenerativeModel('gemini-1.5-flash')

    def run(self, topic: str):
        prompt = f"""
        Role: Smart Research Agent
        Task: Research '{topic}' and draft an outreach email.
        Format: Return ONLY a JSON object matching this schema:
        {{ "research_summary": ["point1", "point2"], "outreach_draft": "email text" }}
        """

        response = self.model.generate_content(
            prompt,
            generation_config={"response_mime_type": "application/json"}
        )
```

```
# Self-Correction Logic
try:
    return AgentOutput.model_validate_json(response.text).model_dump()
except ValidationError:
    # Simple retry logic
    return {"error": "Invalid JSON", "raw": response.text}
```

Step 4: The Interface (app.py)

This creates the web app you can actually use.

Python

```
import streamlit as st
from agent import UniversalTaskAutomator

st.title("🚀 Universal Task Automator")
st.subheader("Nexaverse x American Corner Workshop")

if 'agent' not in st.session_state:
    st.session_state.agent = UniversalTaskAutomator()

topic = st.text_input("What should the agent research?", "AI trends in 2025")

if st.button("Run Agent"):
    with st.spinner("Agent is thinking..."):
        result = st.session_state.agent.run(topic)

        if "error" in result:
            st.error("Agent failed to format data. Try again!")
        else:
            st.write("### 🔍 Research Summary")
            for point in result['research_summary']:
                st.write(f"- {point}")
            st.write("### 📧 Outreach Draft")
            st.info(result['outreach_draft'])
```

Phase 3: Launch & Test

1. **Install:** In the Replit **Shell** (bottom right), type: `pip install -r requirements.txt`
2. **Run:** In the Shell, type: `streamlit run app.py`
3. **Interact:** A new window will open. Enter a topic like *"Impact of AI on local bakeries"* and watch your agent work!

Key Takeaways for Students

- **Prompting is Programming:** Your instructions in `agent.py` are the "code" that drives the brain.
- **Schemas are Safety:** Pydantic ensures the AI doesn't just "talk" but provides data your app can use.
- **Vibe Coding:** You built a complex AI system in minutes by focusing on the "vibe" (intent) rather than heavy syntax.

Built with ❤️ by Nexaverse & American Corner Beirut